

ZyduS Wellness Internal Planning Platform

UI ↔ API Integration Protocol Document

Frontend Consumption Contract — Phase 1 (Authentication & User Administration Surface)

Field	Value
Document Type	UI Integration Protocol (Frontend-facing)
Consumers	React.js / TypeScript frontend team
Companion Document	API Protocol Document (v1.0.0) — authoritative source for every schema reproduced here
Backend	Python FastAPI · MongoDB · Azure
Scope	Every endpoint currently defined in the API Protocol Document: Health, Auth, Admin User APIs, UI User APIs, Reference APIs, Template Generation API
Out of Scope	Sales Planning, COGS Planning, Workflow/Approval, Reports, Consolidation, and Admin Roles/Audit APIs — these are referenced conceptually in the SDD screen catalogue but have no published schema yet. They are not fabricated here; see Section 11 (Gaps).

1. Purpose

This document tells the frontend team, screen by screen, **which endpoint to call, with what payload, expecting what response, and how to render every state** (loading, success, validation error, server error). It is the bridge between:

- [ZyduS_SDD_Volume_II_UIUX](#) — the screen catalogue, wireframes and field specifications, and
- the [API Protocol Document](#) — the authoritative JSON Schema contract already implemented by the backend.

Where the two disagree, **the API Protocol Document wins** (Section 2 explains every place this happens). Nothing in this document invents a new endpoint or field that isn't already in the API Protocol Document — where the SDD implies a capability that has no corresponding schema yet, it is listed as a gap, not guessed at.

2. Reconciliation Notes (SDD vs. Authoritative API)

The SDD screen catalogue was written at a conceptual level before the API was finalized. The items below are corrections the frontend team must apply — they are not contradictions to resolve later, they are settled in favor of the API Protocol Document.

#	SDD says	API Protocol Document says (authoritative)	Frontend impact
1	UI-024 "User Master" calls <code>GET /api/v1/users</code>	Two distinct surfaces exist: Admin User APIs at <code>/api/v1/users</code> (returns <code>UserListResponse</code> wrapper object) and UI User APIs at <code>/api/users</code> (returns a raw <code>UserResponse[]</code> array) — both admin-only	Do not reuse on client method for the response shape differs (wrapped array)
2	UI-027 "Generate Template" calls <code>POST /api/v1/templates/generate</code>	The implemented path is <code>POST /api/v1/templates/generate</code> (no suffix)	Use <code>/api/v1/templates/generate</code> not <code>/api/v1/templates/generate</code>
3	UI-028 "Download Centre" calls <code>GET /api/v1/templates/download/{id}</code>	No such endpoint exists yet. Template generation itself streams the <code>.xlsx</code> binary directly in the <code>POST /api/v1/templates</code> response	Treat Generate Download as one action , not two screens/calls, unless persisted-template endpoint is public
4	Login is described in the SDD (§5.1 Login, Project Bible) as a single-step "Sign In"	The implemented flow is two-step : <code>POST /api/v1/auth/login</code> never returns a session token directly — it returns an OTP challenge (<code>LoginOtpChallengeResponse</code>); the token is only issued after <code>POST /api/v1/auth/otp/verify</code>	The Login screen (UI-001) must include an inline OTP step, username/password
5	The SDD's Screen Inventory (§15.2) does not list a self-registration/sign-up screen anywhere in the Authentication module	The API Protocol Document implements a full self-service signup path: <code>POST /api/v1/auth/users</code> , <code>POST /api/v1/auth/otp/verify</code> (purpose <code>REGISTRATION</code>), resend endpoints, and admin approval (<code>GET /api/v1/users/pending</code> , <code>PUT /api/v1/users/{id}/status</code>)	A Self-Registration screen has no confirmed UI-ID documented here. 000 (Self-Registration ID unconfirmed) with Gauri/BA for official screen name before build
6	UI-051 "Roles & Permissions" implies <code>GET/POST /api/v1/admin/roles</code>	Not present in the current API Protocol Document — <code>/api/roles</code> exists only as a read-	Do not build a Read-Only CRUD screen as

#	SDD says	API Protocol Document says (authoritative)	Frontend impact
		only reference/lookup list (MasterValuesResponse), not a role-management CRUD API	/api/roles; dropdown source

3. Conventions

Item	Value
Base URL	Environment-driven, e.g. https://api.<env>.zyduswellness-planning.azurewebsites.net — never hard-code
Content-Type	application/json for all endpoints except the two noted below
Binary responses	POST /api/v1/templates and GET /api/users/bulk/template return application/vnd.openxmlformats-officedocument.spreadsheetml.sheet (xlsx binary)
Multipart	POST /api/users/bulk/upload only — multipart/form-data
Auth header	Authorization: Bearer <access_token> on every protected call
Trace headers	X-Request-ID, X-Correlation-ID — optional, but the frontend should generate and send both on every call for support/debugging traceability
Error shape	Always { "detail": string } at minimum; treat additionalProperties: true as license for the backend to add fields later without breaking the frontend parser
Date/time	ISO 8601 (date-time) on the wire; render using the SDD's DD-MMM-YYYY display format (Section 9)

4. Global Frontend Integration Patterns

4.1 API Client Layer

A single typed client wraps every call — no screen should call fetch/axios directly.

typescript

```
// api/client.ts
interface ApiError {
  detail: string;
}

class ApiClient {
  constructor(private baseUrl: string, private getToken: () => string | null) {

  private async request<T>(path: string, init: RequestInit = {}): Promise<T> {
    const token = this.getToken();
    const headers: Record<string, string> = {
      "Content-Type": "application/json",
      "X-Request-ID": crypto.randomUUID(),
      "X-Correlation-ID": sessionStorage.getItem("correlationId") ?? crypto.randomBytes(16).toString("hex"),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...(init.headers as Record<string, string> | undefined),
    };
    const res = await fetch(`${this.baseUrl}${path}`, { ...init, headers });
    if (!res.ok) {
      const body: ApiError = await res.json().catch(() => ({ detail: "Unknown error" }));
      throw { status: res.status, detail: body.detail };
    }
    if (res.headers.get("content-type")?.includes("spreadsheetml")) {
      return (await res.blob()) as unknown as T;
    }
    return res.json() as Promise<T>;
  }

  get<T>(path: string) { return this.request<T>(path, { method: "GET" }); }
  post<T>(path: string, body?: unknown) {
    return this.request<T>(path, { method: "POST", body: body ? JSON.stringify(body) : undefined });
  }
  put<T>(path: string, body?: unknown) {
    return this.request<T>(path, { method: "PUT", body: body ? JSON.stringify(body) : undefined });
  }
  delete<T>(path: string) { return this.request<T>(path, { method: "DELETE" }); }
}
```

4.2 Token Storage & Refresh Strategy

Decision point	Recommendation	Rationale
Where to store <code>access_token</code>	In-memory (React context/state), never <code>localStorage</code>	Mitigates XSS token theft

Decision point	Recommendation	Rationale
Where to store <code>refresh_token</code>	<code>httpOnly</code> , <code>Secure</code> , <code>SameSite=Strict</code> cookie set by the backend, if the backend can be extended to do so; otherwise session-scoped encrypted storage as an interim measure	Refresh tokens are long-lived and higher value to an attacker
Refresh trigger	A response interceptor that catches <code>401</code> , calls <code>POST /api/v1/auth/refresh-token</code> once, retries the original request once, and only then redirects to UI-007 (Session Expired) if refresh also fails	Avoids surprise logouts on a single expired access token
Concurrent request race	Queue parallel requests behind a single in-flight refresh call (a shared promise), so 5 simultaneous 401s don't trigger 5 refresh calls	Prevents refresh-token invalidation races
Logout	Always call <code>POST /api/v1/auth/logout</code> before clearing local state, even on client-initiated logout	Server-side session/audit trail closure

4.3 Global Error → UI Message Mapping

HTTP Status	Meaning in this API	Default UI behavior
400	Payload failed validation	Parse <code>detail</code> ; if it names a field, show inline under that field; otherwise show a form-level banner
401	Missing/expired/invalid token	Attempt one silent refresh (4.2); if that fails, route to UI-007 Session Expired
403	Authenticated but not authorized (e.g., non-admin hitting <code>/api/v1/users</code>)	Route to UI-010 Access Denied; do not retry
404	Resource/job/user not found	Toast: "We couldn't find that. It may have been removed."
409	Conflict (duplicate username/email, duplicate signup)	Inline field error: "This email/username is already registered."
500 / 503	Server or dependency (e.g., mail/OTP service) unavailable	Non-dismissible banner: "Something went wrong on our end. Please try again in a few minutes." with a Retry button

4.4 Loading / Empty / Error State Pattern

Every list-fetching or form-submitting screen implements the same four-state pattern; do not invent screen-specific loading UX:

```
idle → loading (skeleton/spinner) → success (data | empty-state)
                                     ↳ error (inline or banner, per 4.3)
```

4.5 RBAC-Driven Conditional Rendering

Role	Can call Admin User APIs (<code>/api/v1/users/*</code>)	Can call UI User APIs (<code>/api/users/*</code>)	Can call Template API	Notes
<code>ADMIN</code>	Yes	Yes	Yes (as an <code>ACTIVE</code> user)	Full administration surface
<code>USER</code> (default self-registered)	No	No	Yes, once <code>ACTIVE</code>	Blocked from admin screens by the router, not just the button
<code>PLANNER</code> , <code>REVIEWER</code> , <code>FINANCE</code> , <code>LEADERSHIP</code>	No	No	Yes, once <code>ACTIVE</code>	Same as <code>USER</code> for this API surface; other roles' screens (Sales Planning, Reports, etc.) are out of scope of this document

Frontend rule: gate both the **route** and the **sidebar entry** on role — a hidden menu item is not a security control, the route guard is. The actual authorization decision is always re-validated server-side (403 handling in 4.3).

4.6 File Download / Upload Handling

Screen	Direction	Pattern
Template Generation	Download	<code>client.post<Blob>('/api/v1/templates', payload)</code> → <code>URL.createObjectURL(blob)</code> → trigger anchor download; filename from <code>Content-Disposition</code> header if present, else a generated default (<code>Zyodus_Template_<cycle>.xlsx</code>)

Screen	Direction	Pattern
Bulk User Template	Download	<code>GET /api/users/bulk/template</code> → same blob pattern, static filename <code>Zyodus_User_Bulk_Template.xlsx</code>
Bulk User Upload	Upload	<code>FormData</code> with <code>file</code> field, <code>Content-Type</code> left unset so the browser sets the multipart boundary; show a progress state; on <code>200</code> , poll <code>GET /api/users/bulk/status/{job_id}</code> (see 6.9)

5. Screen ↔ Endpoint Master Map

Screen	Module (SDD)	Method & Path	Request Schema
UI-000 Self-Registration (<i>ID unconfirmed</i> — see §2.5)	Authentication	<code>POST /api/v1/auth/users</code>	<code>SignupRequest</code>
UI-000a OTP Verification (Registration)	Authentication	<code>POST /api/v1/auth/otp/verify</code>	<code>OTPRequest</code> (purpose= <code>REGISTER</code>)
UI-000b Awaiting Approval (status display)	Authentication	(<i>none</i> — <i>client-side state on</i> <code>status=PENDING_APPROVAL</code>)	—
UI-001 Login (step 1)	Authentication	<code>POST /api/v1/auth/login</code>	<code>LoginRequest</code>
UI-001 Login (step 2 — OTP)	Authentication	<code>POST /api/v1/auth/otp/verify</code>	<code>OTPRequest</code> (purpose= <code>LOGIN</code>)
UI-001 Login — resend OTP	Authentication	<code>POST /api/v1/auth/resend-otp</code> (or <code>/otp</code> , <code>/send-otp</code> — see §6.3 note)	<code>OTPResendRequest</code>
UI-002 Forgot Password	Authentication	<code>POST /api/v1/auth/forgot-password</code>	<code>ForgotPasswordRequest</code>
UI-003 Reset Password	Authentication	<code>POST /api/v1/auth/reset-password</code>	<code>ResetPasswordRequest</code>
UI-004 Change Password	Authentication	<code>POST /api/v1/auth/change-password</code>	<code>ChangePasswordRequest</code>

Screen	Module (SDD)	Method & Path	Request Schema
UI-005a MFA (Future Phase)	Authentication	(no endpoint — out of Phase 1 scope)	—
UI-006 Account Locked	Authentication	(display-only; triggered by a 401/403 from any call)	—
UI-007 Session Expired	Authentication	<code>POST /api/v1/auth/refresh-token</code> (attempted first)	<code>RefreshTokenRec</code>
UI-008 User Profile	Authentication	<code>GET /api/v1/auth/users/me</code>	—
UI-008 Logout action	Authentication	<code>POST /api/v1/auth/logout</code>	—
UI-009 Active Sessions	Authentication	No endpoint published — Gap, see §11	—
UI-010 Access Denied	Authentication	(display-only; triggered by 403)	—
UI-024 User Master — list (admin console)	Master Data / Admin	<code>GET /api/v1/users</code>	—
UI-024 User Master — pending queue	Master Data / Admin	<code>GET /api/v1/users/pending</code>	—
UI-024 User Master — approve/reject	Master Data / Admin	<code>PUT /api/v1/users/{user_id}/status</code>	<code>AdminUserStatus</code>
UI-024 User Master — create user	Master Data / Admin	<code>POST /api/v1/users</code>	<code>AdminCreateUser</code>
UI-024 User Master — delete user	Master Data / Admin	<code>DELETE /api/v1/users/{user_id}</code>	—
UI-024b User List (UI-facing grid)	Master Data / Admin	<code>GET /api/users</code>	—
UI-024b Create user (UI-facing	Master Data / Admin	<code>POST /api/users</code>	<code>UserCreateReque</code>

Screen	Module (SDD)	Method & Path	Request Schema
form)			
UI-024b View single user	Master Data / Admin	GET /api/users/{user_id}	—
UI-024b Delete user (UI-facing)	Master Data / Admin	DELETE /api/users/{user_id}	—
UI-024c Bulk User Upload	Master Data / Admin	POST /api/users/bulk/upload	multipart/form-
UI-024c Bulk Template Download	Master Data / Admin	GET /api/users/bulk/template	—
UI-024c Bulk Job Status	Master Data / Admin	GET /api/users/bulk/status/{job_id}	—
UI-024c Bulk Job Errors	Master Data / Admin	GET /api/users/bulk/errors/{job_id}	—
Registration/User-Master dropdowns	Reference Data	GET /api/segments, /api/channels, /api/brands, /api/roles	—
UI-027 Generate Template (<i>path corrected — §2.2</i>)	Template Management	POST /api/v1/templates	GenerateTemplat
System/App health check	Platform	GET /health	—

6. Per-Screen Protocols

6.1 UI-000 Self-Registration (*screen ID unconfirmed*)

Purpose: New employees create their own account, which lands in `PENDING_OTP` then `PENDING_APPROVAL` until an Admin activates it.

Form fields → JSON property:

UI Field	JSON Property	Client-side validation (mirrors backend)
First Name	<code>first_name</code>	required, 1-50 chars
Last Name	<code>last_name</code>	required, 1-50 chars
Username	<code>username</code>	required, 3-50 chars
Work Email	<code>email</code>	required, valid email format
Password	<code>password</code>	required, 12-128 chars — see §9 for the full policy
Segment(s)	<code>segment</code>	optional multi-select, sourced from <code>GET /api/segments</code>
Channel(s)	<code>channel</code>	optional multi-select, sourced from <code>GET /api/channels</code>
Brand(s)	<code>brand</code>	optional multi-select, sourced from <code>GET /api/brands</code>

`role` is fixed to `"USER"` by the schema (`enum: ["USER"]`) — do not expose a role picker on this screen.

Flow:

```

Fill form → POST /api/v1/auth/users
  → 201 OTPVerifyResponse { success, message, status: "PENDING_OTP" }
  → Show OTP entry screen (UI-000a), pre-fill email, start resend-cooldown timer
→ Enter 6-digit OTP → POST /api/v1/auth/otp/verify { email, purpose: "REGISTRATION", otp }
  → 200 SignupResponse { ..., status: "PENDING_APPROVAL" }
  → Show UI-000b "Your account is awaiting administrator approval"

```

Error states:

Status	Cause	UI Response
409	Email or username already registered	Inline error on the relevant field
503	OTP/mail dispatch failed	Banner: "We couldn't send your verification code — please try again shortly"
400 (on verify)	Wrong/expired OTP	Inline error under OTP field, keep resend available
403 (on verify)	OTP purpose/context mismatch	Restart the registration flow
404 (on verify)	No pending challenge for this email	Restart the registration flow

6.2 UI-024 / UI-024b User Master — Admin Approval Queue

Purpose: Admin reviews `PENDING_APPROVAL` users and activates or rejects them, and manages the full user roster.

Key behaviors:

- The **pending queue** view calls `GET /api/v1/users/pending` and renders `UserListResponse.users`.
- The **full roster** view has two possible data sources depending on which screen variant is built (§2.1) — do not mix them:
 - Admin console variant → `GET /api/v1/users` → `UserListResponse.users`
 - UI-facing grid variant → `GET /api/users` → a bare `UserResponse[]`
- **Approve/Reject** action: `PUT /api/v1/users/{user_id}/status` with body:

```
json
{
  "status": "ACTIVE",
  "segment": ["Nutrition"],
  "channel": ["GT", "MT"],
  "brand": ["Complan"],
  "permissions": ["sales_plan.edit"]
}
```

`status` only accepts `"ACTIVE"` or `"REJECTED"` at this endpoint — any other transition (e.g., to `INACTIVE`) is out of scope of this action and must go through a different flow if/when published.

- **Create user directly** (skips self-registration/OTP): `POST /api/v1/users` (admin variant, `AdminCreateUserRequest`, defaults `role: USER`, `status: ACTIVE`) or `POST /api/users` (UI variant, `UserCreateRequest`, which **requires** non-empty `segment`, `channel`, `brand`, and `role` — no defaults).
- **Delete**: `DELETE /api/v1/users/{user_id}` or `DELETE /api/users/{user_id}` depending on variant; both return `AdminDeleteUserResponse`.

Client-side validation for the Create-User form (UI variant, stricter):

Field	Rule
<code>segment</code> , <code>channel</code> , <code>brand</code>	<code>minItems: 1</code> each — the Save button stays disabled until all three have at least one selection
<code>role</code>	required, one of <code>ADMIN</code> , <code>USER</code> , <code>PLANNER</code> , <code>REVIEWER</code> , <code>FINANCE</code> , <code>LEADERSHIP</code>
<code>password</code>	12-128 chars, same policy as registration

6.3 UI-001 Login

Two-step flow (see §2.4):

Step 1 – Credentials

```
POST /api/v1/auth/login { identifier, password, role? }
→ 200 LoginOtpChallengeResponse { challenge_id, purpose: "LOGIN", email, expires_at, resend_available_at }
→ Navigate to inline OTP panel; start countdown to expires_at
```

Step 2 – OTP

```
POST /api/v1/auth/otp/verify { email, purpose: "LOGIN", otp }
→ 200 LoginResponse { access_token, token_type, expires_in, refresh_token, refresh_expires_in }
→ Store tokens (§4.2); GET /api/v1/auth/users/me to hydrate profile/role; route by role per the Project Bible's post-login redirect table
```

Resend OTP: three equivalent endpoints exist in the API Protocol Document —

`/api/v1/auth/otp`, `/api/v1/auth/send-otp`, `/api/v1/auth/resend-otp` — all take `OTPResendRequest` and return `OTPVerifyResponse`. **Pick exactly one for this screen**

(recommend `/resend-otp` for clarity) and confirm with backend which is the canonical one; do not call more than one interchangeably across the app.

Error states:

Status	Cause	UI Response
401 (login)	Bad credentials	Inline: "Incorrect username or password" — never say which field is wrong
403 (login)	Account not <code>ACTIVE</code> (e.g., still <code>PENDING_APPROVAL</code> , <code>REJECTED</code> , <code>INACTIVE</code>)	Show account-status-specific message; if <code>PENDING_APPROVAL</code> , offer link to UI-000b
503 (login)	Downstream auth dependency down	Banner per §4.3
400/403/404/409 (otp verify)	Same as §6.1's OTP error table	Same handling

6.4 UI-002 Forgot Password

```
POST /api/v1/auth/forgot-password { email }
→ 200 OTPVerifyResponse
→ Show neutral confirmation: "If that email is registered, a reset code has been sent."
```

Do not reveal whether the email exists in error responses — treat 400/503 as the only distinguishable states; anything else should look identical to the success path for account-enumeration safety, even though the schema technically allows more granular errors.

6.5 UI-003 Reset Password

```
POST /api/v1/auth/reset-password { email, otp, new_password }
→ 200 OTPVerifyResponse → redirect to UI-001 Login with a success toast
```

Client-side password policy mirrors §9 exactly, plus a live strength meter as specified in the SDD wireframe (visual only — server is the source of truth for acceptance).

6.6 UI-004 Change Password

```
POST /api/v1/auth/change-password { current_password, new_password } [Bearer required]
→ 200 OTPVerifyResponse
```

6.7 UI-007 Session Expired

Not a screen the user navigates to directly — it is the terminal state of the refresh flow in §4.2:

```
Any protected call → 401
  → POST /api/v1/auth/refresh-token { refresh_token }
    → 200 LoginResponse → retry original call transparently
    → 401/403 → clear all local state → show UI-007 → "Login Again" → UI-001
```

6.8 UI-008 User Profile

```
GET /api/v1/auth/users/me [Bearer required]
→ 200 UserResponse
```

Renders `first_name`, `last_name`, `email`, `role`, `status`, `segment[]`, `channel[]`, `brand[]`, `permissions[]`. There is **no update-profile endpoint** in the current API Protocol Document — the SDD's "Edit Profile (if permitted)" button has no backing API yet; disable or hide that button until one is published (see §11).

Logout button on this screen: `POST /api/v1/auth/logout` → clear tokens → route to UI-001.

6.9 UI-024c Bulk User Upload

1. GET `/api/users/bulk/template` → download `xlsx` (§4.6)
2. Admin fills template offline
3. POST `/api/users/bulk/upload` (multipart: file)
→ 200 BulkUploadJobResponse { `job_id`, `status`: "PENDING" | "PROCESSING", ... }
4. Poll GET `/api/users/bulk/status/{job_id}` every 2-3s while status is PENDING/PROCESSING
→ on COMPLETED or COMPLETED_WITH_ERRORS, stop polling
5. If COMPLETED_WITH_ERRORS or `failed_rows > 0`:
GET `/api/users/bulk/errors/{job_id}` → BulkUploadErrorsResponse.errors[]
→ render as a table: `row_number`, `field`, `message`, `raw_value`

Render `processed_rows / total_rows` as a progress bar while polling; show `successful_rows` vs `failed_rows` as the completion summary.

6.10 UI-027 Generate Template (path corrected, §2.2)

```
POST /api/v1/templates [Bearer required, status must be ACTIVE]
  { planning_cycle?, planning_years?, template_version?, workbook_version? }
→ 200, binary xlsx → trigger browser download (§4.6)
```

Status	Cause	UI Response
400	Invalid cycle/year combination	Inline form error
401	Not authenticated	Route to UI-001
403	Authenticated but account not <code>ACTIVE</code>	Explain account status; do not imply a permissions problem
500	Template rendering failure server-side	Banner + Retry, per §4.3

6.11 Reference Data Dropdowns

`GET /api/segments`, `/api/channels`, `/api/brands`, `/api/roles` are unauthenticated, cacheable `MasterValuesResponse` lists (`{ items: string[] }`). Fetch once per session on app bootstrap (or lazily on first dropdown open) and cache in a shared store — these do not need to be re-fetched per screen. They back the multi-selects in Self-Registration, User Master create/edit, and the Admin approval form's segment/channel/brand assignment.

7. TypeScript Type Definitions

Drop-in types generated from the API Protocol Document's JSON Schema components, for direct use in the API client and form models.

typescript


```
// types/enums.ts
export type UserStatus = "PENDING_OTP" | "PENDING_APPROVAL" | "ACTIVE" | "REJECT"
export type OTPPurpose = "REGISTRATION" | "LOGIN" | "PASSWORD_RESET";
export type Role = "ADMIN" | "USER" | "PLANNER" | "REVIEWER" | "FINANCE" | "LEADER"
export type BulkUploadJobStatus = "PENDING" | "PROCESSING" | "COMPLETED" | "COMPLETED_FAILED"

// types/auth.ts
export interface SignupRequest {
  first_name: string;
  last_name: string;
  username: string;
  email: string;
  password: string;
  role?: "USER";
  segment?: string[] | null;
  channel?: string[] | null;
  brand?: string[] | null;
}

export interface OTPRequest {
  email: string;
  purpose: OTPPurpose;
  otp: string;
}

export interface OTPResendRequest {
  email: string;
  purpose: OTPPurpose;
}

export interface OTPVerifyResponse {
  success: boolean;
  message: string;
  status?: UserStatus | null;
}

export interface SignupResponse {
  id: string;
  first_name: string;
  last_name: string;
  username: string;
  email: string;
  role: Role;
  status: UserStatus;
  created_at: string;
}
```

```
export interface LoginRequest {
  identifier: string;
  password: string;
  role?: Role | null;
}

export interface LoginOtpChallengeResponse {
  challenge_id: string;
  purpose: OTPPurpose;
  email: string;
  expires_at: string;
  resend_available_at?: string | null;
}

export interface LoginResponse {
  access_token: string;
  token_type: string;
  expires_in: number;
  refresh_token?: string | null;
  refresh_expires_in?: number | null;
}

export interface RefreshTokenRequest {
  refresh_token: string;
}

export interface ForgotPasswordRequest {
  email: string;
}

export interface ResetPasswordRequest {
  email: string;
  otp: string;
  new_password: string;
}

export interface ChangePasswordRequest {
  current_password: string;
  new_password: string;
}

export interface APIMessageResponse {
  success: boolean;
  message: string;
  data?: Record<string, unknown> | null;
}
```

```
// types/user.ts
export interface UserResponse {
  id: string;
  first_name: string;
  last_name: string;
  username: string;
  email: string;
  role: Role;
  status: UserStatus;
  segment: string[];
  channel: string[];
  brand: string[];
  permissions: string[];
  created_at?: string | null;
  updated_at?: string | null;
}

export interface UserListResponse {
  users: UserResponse[];
}

export interface AdminUserStatusUpdateRequest {
  status: "ACTIVE" | "REJECTED";
  segment?: string[] | null;
  channel?: string[] | null;
  brand?: string[] | null;
  permissions?: string[] | null;
}

export interface AdminCreateUserRequest {
  first_name: string;
  last_name: string;
  username: string;
  email: string;
  password: string;
  segment?: string[] | null;
  channel?: string[] | null;
  brand?: string[] | null;
  permissions?: string[];
  role?: Role;
  status?: UserStatus;
}

export interface AdminDeleteUserResponse {
  success: boolean;
  message: string;
}
```

```
}

export interface UserCreateRequest {
  first_name: string;
  last_name: string;
  username: string;
  email: string;
  password: string;
  segment: string[]; // minItems: 1
  channel: string[]; // minItems: 1
  brand: string[]; // minItems: 1
  role: Role;
}

// types/masters.ts
export interface MasterValuesResponse {
  items: string[];
}

// types/bulk.ts
export interface BulkUploadErrorItem {
  row_number: number;
  field?: string | null;
  message: string;
  raw_value?: string | null;
}

export interface BulkUploadJobResponse {
  job_id: string;
  status: BulkUploadJobStatus;
  total_rows: number;
  processed_rows: number;
  successful_rows: number;
  failed_rows: number;
  created_at: string;
  completed_at?: string | null;
}

export interface BulkUploadErrorsResponse {
  job_id: string;
  status: BulkUploadJobStatus;
  errors: BulkUploadErrorItem[];
}

// types/templates.ts
export interface GenerateTemplateRequest {
  planning_cycle?: string | null;
```

```
planning_years?: number[] | null;
template_version?: string | null;
workbook_version?: string | null;
}
```

8. State Machines Relevant to the UI

8.1 User Status Lifecycle (drives which screen a user sees after login/registration)

```
PENDING_OTP → (OTP verified) → PENDING_APPROVAL → (Admin: ACTIVE) → ACTIVE
                                                    ↳ (Admin: REJECTED) →
REJECTED
ACTIVE → (Admin action, no direct endpoint yet) → INACTIVE [see Gap in §11]
```

Status	Screen shown on login attempt
PENDING_OTP	Redirect back into the OTP verification step (UI-000a)
PENDING_APPROVAL	UI-000b "Awaiting Approval"
ACTIVE	Normal login proceeds to role-based landing page
REJECTED	Account-rejected message with support contact; no retry
INACTIVE	Account-disabled message with support contact; no retry

8.2 Bulk Upload Job Lifecycle

```
PENDING → PROCESSING → COMPLETED
                        ↳ COMPLETED_WITH_ERRORS (partial success – show error
table)
                        ↳ FAILED (show generic failure +
retry-upload option)
```

9. Client-Side Validation Rules (Mirrors Backend — Never a Substitute For It)

Field	Rule	Applies to
first_name, last_name	1–50 chars, required	Signup, Admin create user

Field	Rule	Applies to
<code>username</code>	3-50 chars, required	Signup, Admin create user
<code>email</code>	Valid email format, required	Signup, Login identifier, Forgot/Reset password
<code>password</code> / <code>new_password</code>	12-128 chars, required	Signup, Reset, Change password, Admin create user
<code>otp</code>	Exactly 6 characters	All OTP verification steps
<code>segment</code> / <code>channel</code> / <code>brand</code> (UI User API create)	<code>minItems: 1</code> each	UI-facing Create User form only — the Admin API variant allows these to be null

All client-side checks exist purely for immediate feedback and to reduce round-trips; the 400 response from the server is always the final authority, and its `detail` message should be surfaced verbatim (or mapped per §4.3) rather than re-derived client-side.

10. Sequence Summaries (Text Flow, Consistent With SDD Journey Notation)

Self-Registration → Activation:

```
Fill Signup Form -> POST /auth/users -> Enter OTP -> POST /auth/otp/verify
-> Awaiting Approval screen -> (Admin) PUT /users/{id}/status ACTIVE
-> Email/notification (server-side) -> User can now Login
```

Login → Working Session → Expiry:

```
Enter credentials -> POST /auth/login -> Enter OTP -> POST /auth/otp/verify
-> tokens stored -> GET /auth/users/me -> role-based landing page
-> [later] any call returns 401 -> POST /auth/refresh-token (silent)
-> success: retry original call | failure: UI-007 Session Expired ->
Login Again
```

Admin Bulk User Provisioning:

```
GET /users/bulk/template -> fill offline -> POST /users/bulk/upload
-> poll GET /users/bulk/status/{job_id} -> COMPLETED_WITH_ERRORS
-> GET /users/bulk/errors/{job_id} -> fix file -> re-upload
```

11. Assumptions & Open Items (Flagged, Not Guessed)

#	Item	Why it matters	Recommended next step
1	Self-Registration screen has no confirmed UI-ID in the SDD screen catalogue	Build team needs a formal screen number for traceability and QA test-case mapping	Confirm with BA/Gauri; suggest reserving UI-011 or UI-005b
2	UI-009 Active Sessions has no backing endpoint anywhere in the API Protocol Document	The SDD wireframe implies device/IP/session listing and remote logout, which requires session-store visibility not yet exposed	Flag to backend team as a Phase 1 gap or defer to Phase 2
3	UI-008 "Edit Profile" button has no update-profile endpoint (<code>GET /auth/users/me</code> is read-only)	Button should be hidden/disabled, not wired to a non-existent call	Confirm scope: is self-service profile edit in Phase 1 at all?
4	Three interchangeable resend-OTP endpoints (<code>/otp</code> , <code>/send-otp</code> , <code>/resend-otp</code>)	Risk of inconsistent usage across screens/teams if not standardized	Backend to confirm canonical endpoint; deprecate the other two in a future API version
5	No endpoint to transition an <code>ACTIVE</code> user to <code>INACTIVE</code> (only <code>ACTIVE</code> / <code>REJECTED</code> via the status-update endpoint)	Admin cannot deactivate an existing active user through this API surface	Confirm whether deactivation is handled via <code>DELETE</code> (soft-delete) or is a genuine gap
6	<code>POST /api/v1/templates</code> returns the file directly with no persisted "download later" record	UI-028 Download Centre (list of previously generated templates) has no API to list historical generations	Confirm whether Download Centre is Phase 1 scope; if so, a <code>GET /api/v1/templates</code> (list) endpoint is needed
7	Admin User APIs (<code>/api/v1/users</code>) and UI User APIs (<code>/api/users</code>) appear functionally duplicative	Two parallel code paths increase maintenance and bug-fix surface for the frontend	Recommend confirming with backend whether one is deprecated/legacy before building both screen variants

12. Quick Reference — Full Endpoint Index

Method	Path	Auth	Primary Screen(s)
GET	<code>/health</code>	No	Platform/ops only
POST	<code>/api/v1/auth/users</code>	No	UI-000
POST	<code>/api/v1/auth/otp/verify</code>	No	UI-000a, UI-001
POST	<code>/api/v1/auth/verify-otp</code>	No	Alias of above — confirm canonical
POST	<code>/api/v1/auth/otp</code>	No	Resend (§6.3 note)
POST	<code>/api/v1/auth/send-otp</code>	No	Resend (§6.3 note)
POST	<code>/api/v1/auth/resend-otp</code>	No	Resend (§6.3 note)
POST	<code>/api/v1/auth/login</code>	No	UI-001
POST	<code>/api/v1/auth/refresh-token</code>	No	UI-007 (silent)
POST	<code>/api/v1/auth/forgot-password</code>	No	UI-002
POST	<code>/api/v1/auth/reset-password</code>	No	UI-003
POST	<code>/api/v1/auth/change-password</code>	Bearer	UI-004
GET	<code>/api/v1/auth/users/me</code>	Bearer	UI-008
POST	<code>/api/v1/auth/logout</code>	Bearer	UI-008
GET	<code>/api/v1/users/pending</code>	Bearer (admin)	UI-024
GET	<code>/api/v1/users</code>	Bearer (admin)	UI-024
PUT	<code>/api/v1/users/{user_id}/status</code>	Bearer (admin)	UI-024
POST	<code>/api/v1/users</code>	Bearer (admin)	UI-024
DELETE	<code>/api/v1/users/{user_id}</code>	Bearer (admin)	UI-024

Method	Path	Auth	Primary Screen(s)
GET	/api/users	Bearer (admin)	UI-024b
GET	/api/users/bulk/template	Bearer (admin)	UI-024c
POST	/api/users	Bearer (admin)	UI-024b
POST	/api/users/bulk/upload	Bearer (admin)	UI-024c
GET	/api/users/bulk/status/{job_id}	Bearer (admin)	UI-024c
GET	/api/users/bulk/errors/{job_id}	Bearer (admin)	UI-024c
GET	/api/users/{user_id}	Bearer (admin)	UI-024b
DELETE	/api/users/{user_id}	Bearer (admin)	UI-024b
GET	/api/segments	No	Dropdowns
GET	/api/channels	No	Dropdowns
GET	/api/brands	No	Dropdowns
GET	/api/roles	No	Dropdowns
POST	/api/v1/templates	Bearer (ACTIVE)	UI-027

END OF UI ↔ API INTEGRATION PROTOCOL DOCUMENT